Binary Search Tree (BST)

- A search tree DS that supports each of the dynamic set operations : search, minimum, maximum, predecessor, successor, insert, delete.
- Basic operations take time proportional to the height of the tree

> Height = # edges from topmost node to the farthest node in a tree

→ for a complete binary tree of n nodes, the operations run in $\Theta(\lg n)$ worst-case time.

- A BST is organized in a Binary tree.
- Can be represented with a linked data structure containing nodes with fields : key, left, right, p

Points to: left child, right child, Parent

e.g.:

```
Struct Node {
    int key;
    Node * left;
    Node * right;
    Node * p;
};
```

→ If a child or parent for a node is missing, the attribute contains NIL

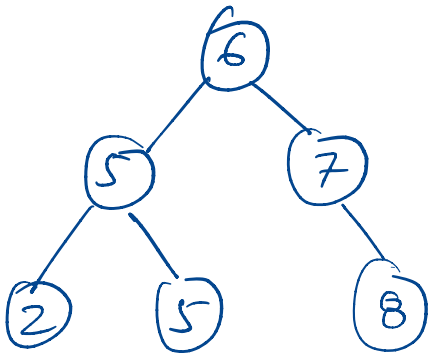
- Tree has a root attribute that points to the root node, or NIL if the tree is empty.

— The root node T.root is the only node whose parent is NIL.

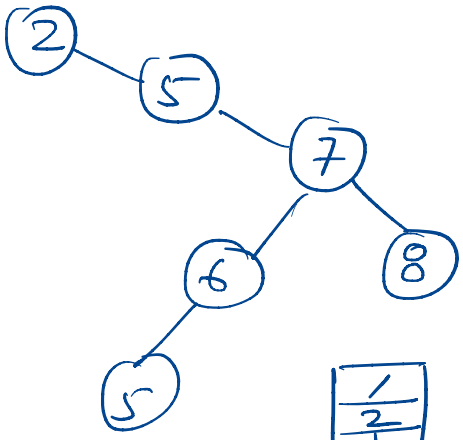
— keys in a BST are stored to satisfy the Binary-search-tree property:

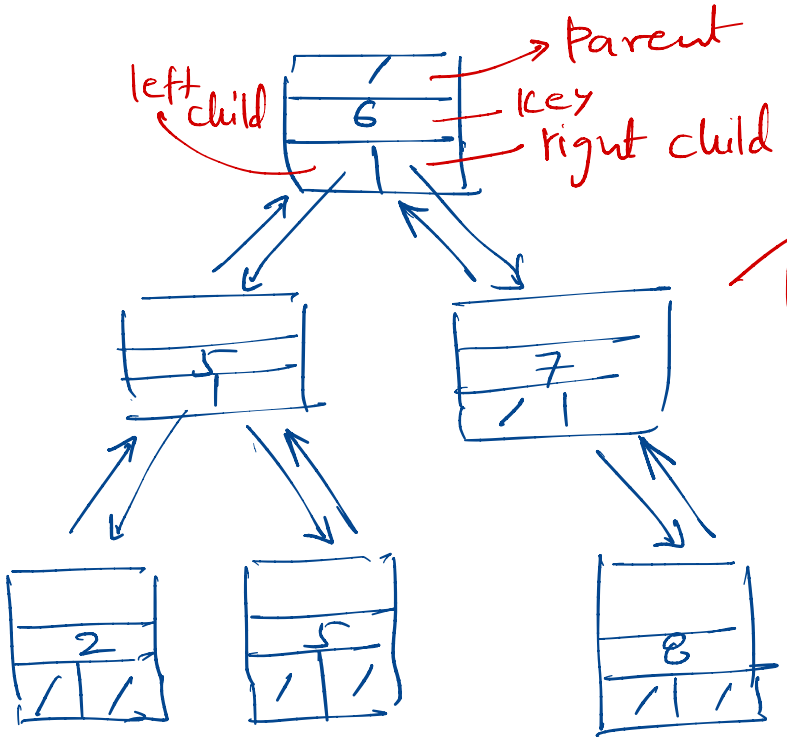
" Let x be a node in a BST. If y is a node in the left subtree of x, then $y.key \leq x.key$. If y is a node in the right subtree of x, then $y.key \geq x.key$. "

Examples:

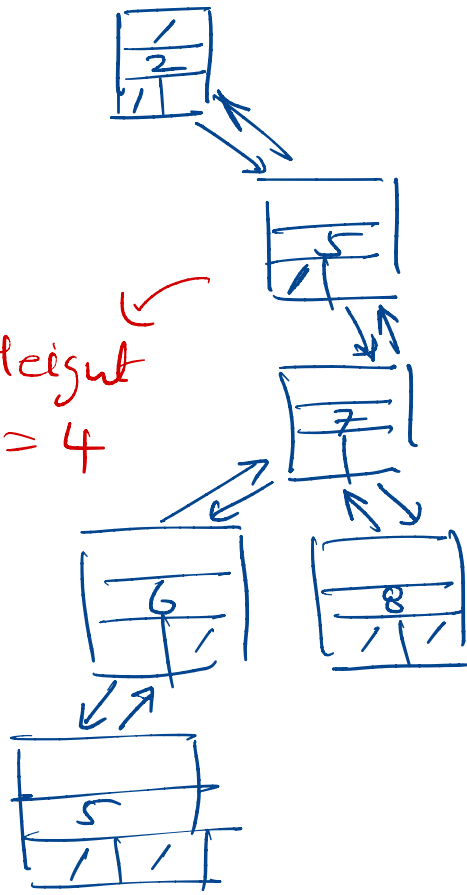


Same set of values





Height = 2



Height = 4

BST traversal

→ Digression →

Tree traversals:

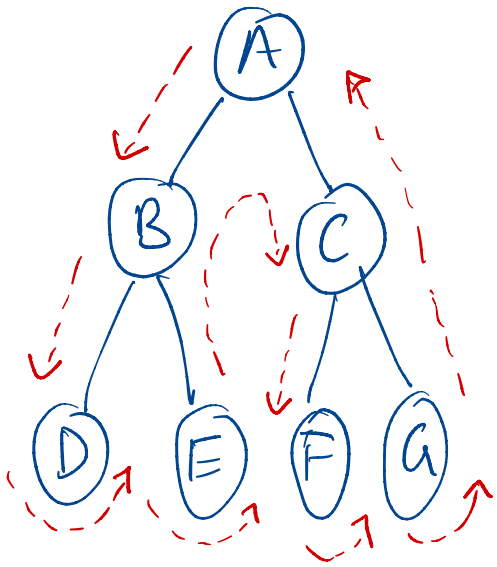
→ Depth First Search (DFS)

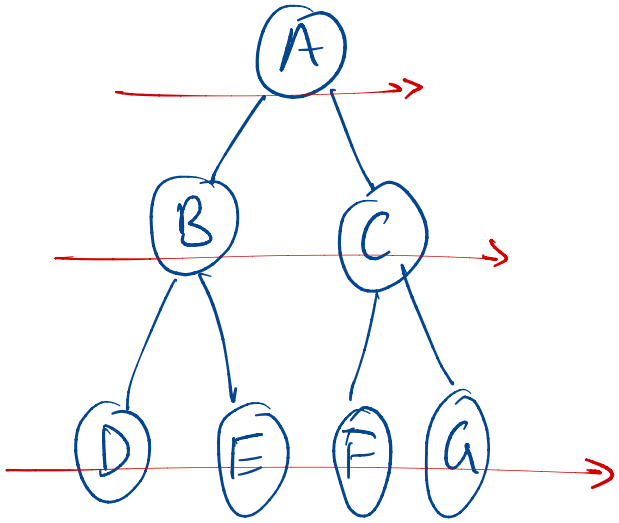
Explore the tree in depth before going to next sibling

→ Breadth First Search (BFS)

(or level-order search)

The tree is broadened as much as possible before going to the next level





DFS: Pre: A B D E C F G

In: D B E A F C G

Post: D E B F G C A

BFS: A B C D E F G

DFS:

a) Pre-order traversal

root left right

walk order: TLR

- visit root first
- recursively traverse the left subtree
- recursively traverse the right subtree

b) In-order traversal

walk order: LTR

- recursively traverse the left subtree
- visit the root
- recursively traverse the right subtree

c) Post-order traversal

walk order: LRT

- recursively traverse left subtree
- recursively " right "
- visit the root

— back —>

BST traversal

- It is possible to print all the keys in a BST in sorted order by an Inorder tree walk

Due to the Property of BST

Inorder tree walk ⇓ LTR order

```
Inorder-Tree-Walk(x)                    // The root of a subtree of T
if x ≠ NIL
    Inorder-Tree-Walk(x.left)           // walk the left subtree
    Print x.key                         // visit the root
    Inorder-Tree-Walk(x.right)          // walk the right subtree
```

It takes $\theta(n)$ time to walk an n-node BST.

- Querying a BST : min/max, successor/Predecessor, search

- search : returns a pointer to node with given key k if one exists in the subtree else return NIL

Tree-Search (x, k)

```
if x == NIL or k == x.key
    return x
if k < x.key
    return Tree-Search (x.left, k)
else
    return Tree-Search (x.right, k)
```

If searching the entire tree, then call:

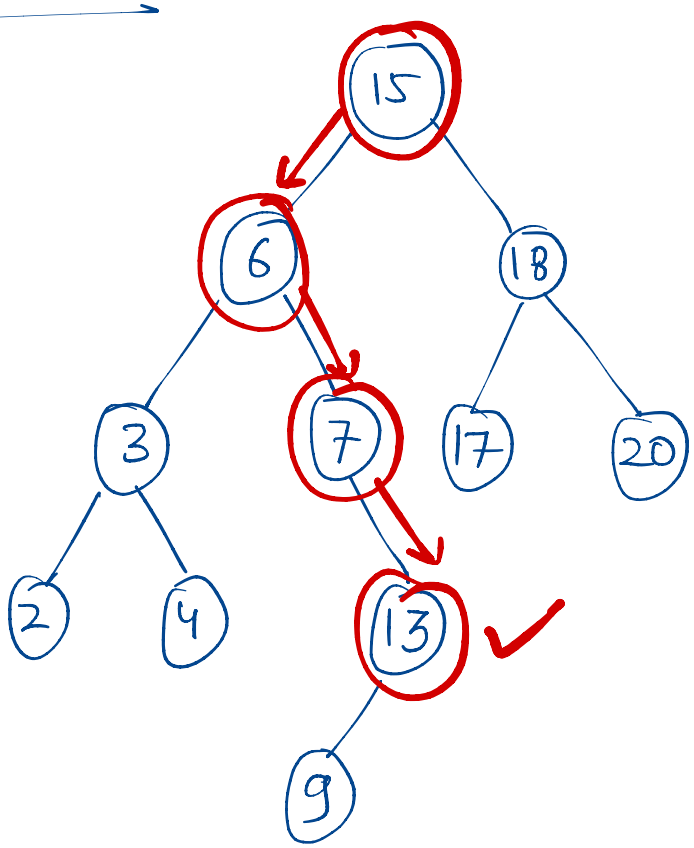
Tree-Search (T.root, k)

Iterative-Tree-Search (x, k)

```
While x ≠ NIL and k ≠ x.key
    if k < x.key
        x = x.left
    else
        x = x.right
return x
```

Running time O(h) ↓ height of the tree

Example Search:



Search 1: 13

Search 2: 1

- Minimum / Maximum

↳ returns pointer to node containing the min/max key in the subtree rooted at a given node x (assumed to be non-nil)

```
Tree-Minimum(x)
while x.left ≠ NIL
    x = x.left
return x
```

```
Tree-Maximum(x)
while x.right ≠ NIL
    x = x.right
return x
```

- Successor / Predecessor

↳ Find successor (of a given node) in the sorted order determined by inorder walk.

↳ Successor of node x = node with smallest value greater than (or equal to) $x.key$.

- Define successor as the next node visited in an inorder tree walk.

- Two cases:

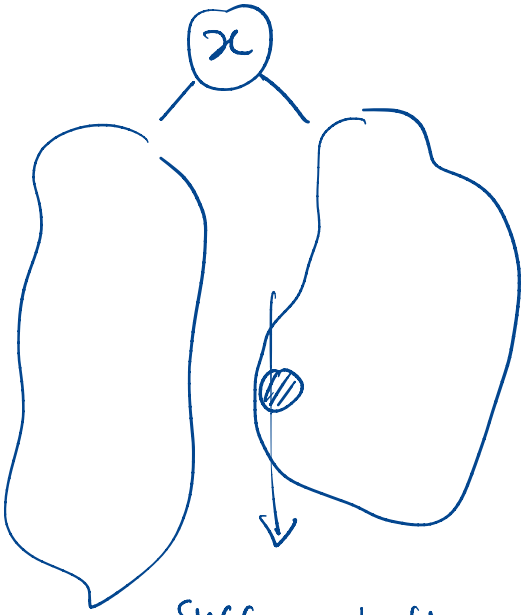
(i) If the right subtree of node x is non-empty

↳ Successor = leftmost node in x's right subtree

↓

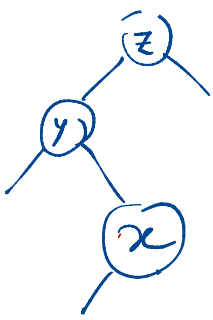
Call:

Tree-Minimum($x.right$)

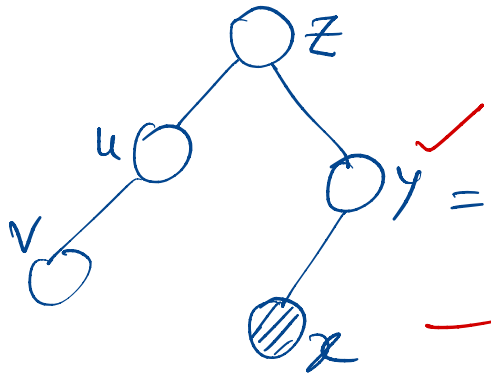


succ. = leftmost node

(ii) If the right subtree of node x is empty



Then z is the successor, if z is the lowest ancestor of x whose left child is also an ancestor of x.



y = succ. of x

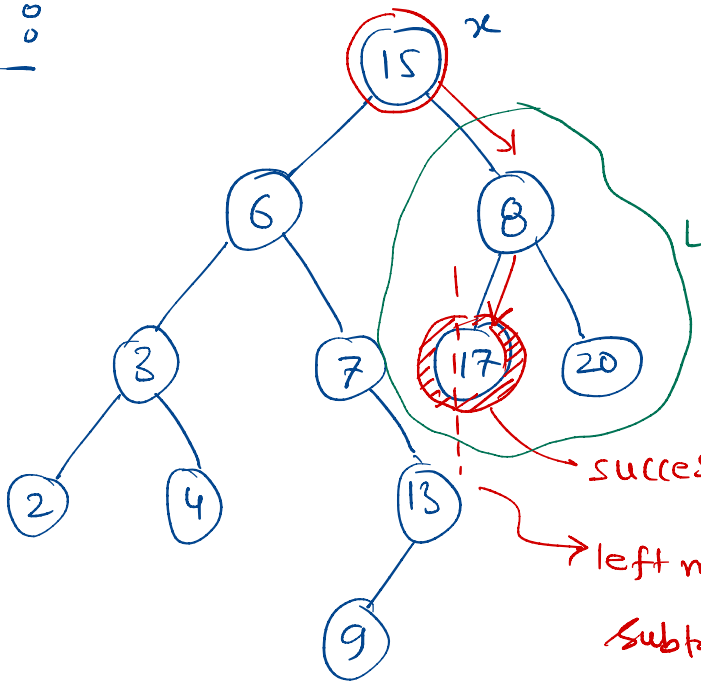
→ x is also an ancestor of x !

i.e, Definition of ancestor is reflexive

Tree-Successor (x)

```
if x.right ≠ NIL
        return Tree-Minimum (x.right)
else
    | y = x.p
    | while y ≠ NIL and x == y.right
    |     | x = y
    |     | y = y.p
    | return y
```

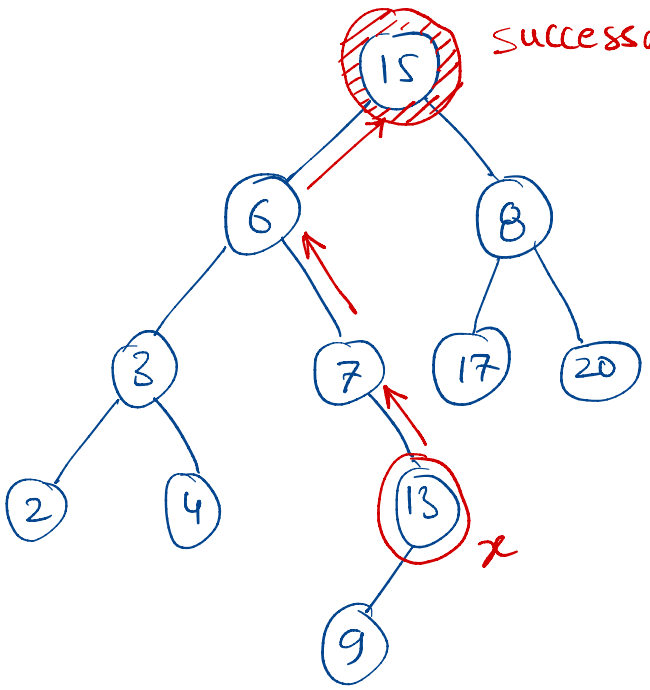
Examples :



Left subtree of x

successor (x)

left most node in right subtree ⇒ Case 1



successor (x) : an ancestor of x whose left child is also an ancestor of x

⇒ Case 2

Insertion :

→ It causes the dynamic tree to change.

→ The tree needs to be modified s.t. that the property continues to hold

→ Inserts a new node into BST (at an appropriate leaf)

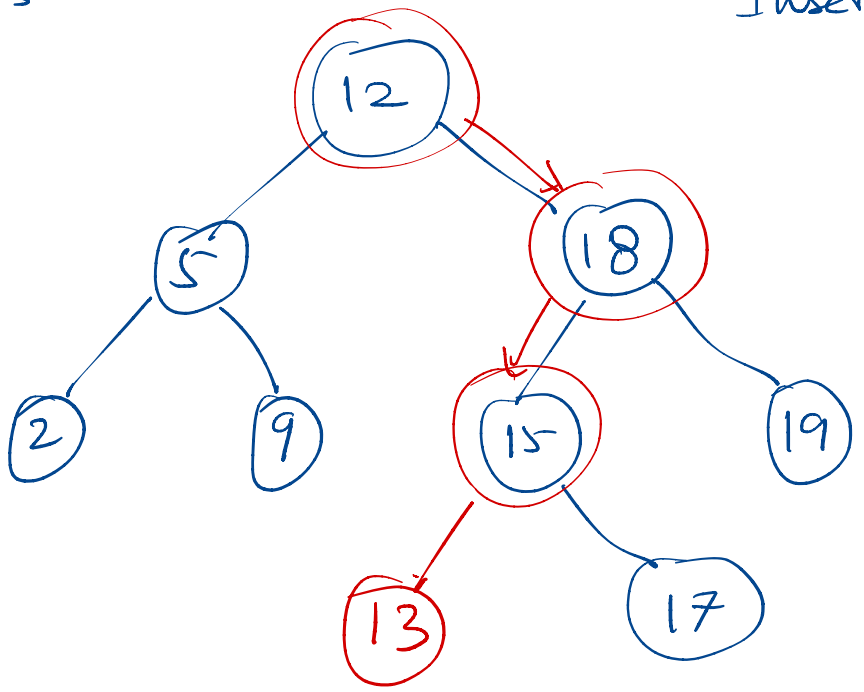
Input: a BST node z for which $z.key$ has been set and $z.left = NIL$, $z.right = NIL$

Tree-Insert (T, z)

```
x = T.root        // Node being compared with z
y = NIL
while x ≠ NIL     // Descend until reaching a leaf
    y = x
    if z.key < x.key
        x = x.left
    else x = x.right
z.p = y           // location found! Insert z with Parent y
if y == NIL
        T.root = z    // tree was empty
else if z.key < y.key
        y.left = z
else    y.right = z
```

Example :

Insert : 13



Deletion : Delete a node from the BST

↳ 3 basic cases :

(i) If z has no children, simply remove it by modifying its its parent to replace z by NIL

(ii) If z has just 1 child, then elevate that child to take z's position in the tree by modifying z's parent to replace z by z's child.

(iii) If z has 2 children, find z's in-order successor y - which must belong to z's right subtree, and move y to take z's position in the tree

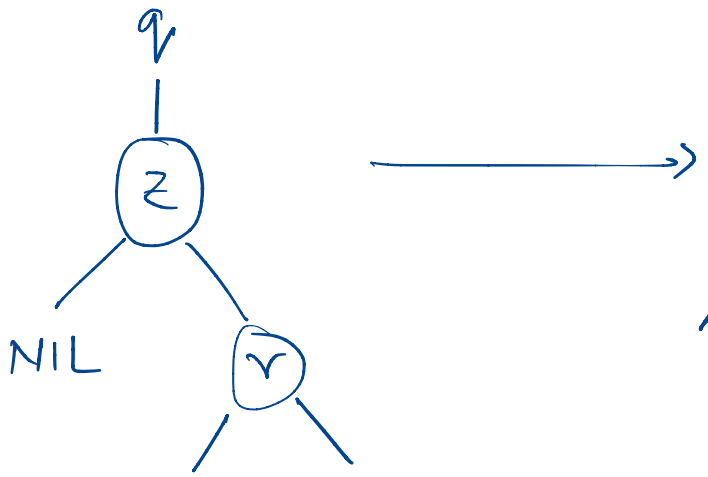
...

The rest of z's original right subtree becomes y's new right subtree, and z's left subtree becomes y's new left subtree

↳ Note: since y is z's successor, it cannot have a left subtree

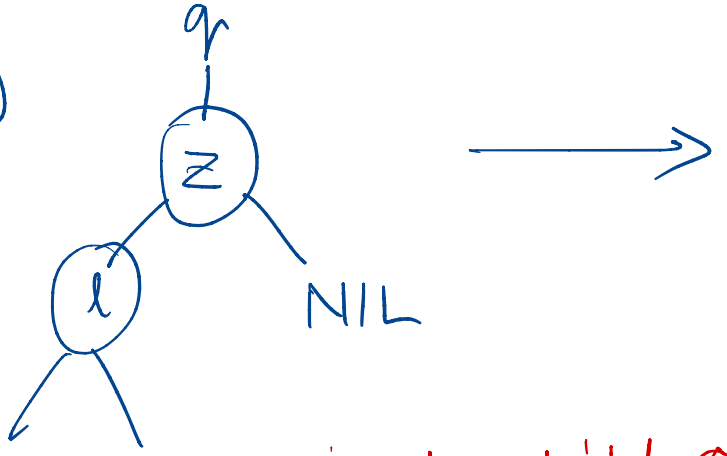
Illustrations:

(a)



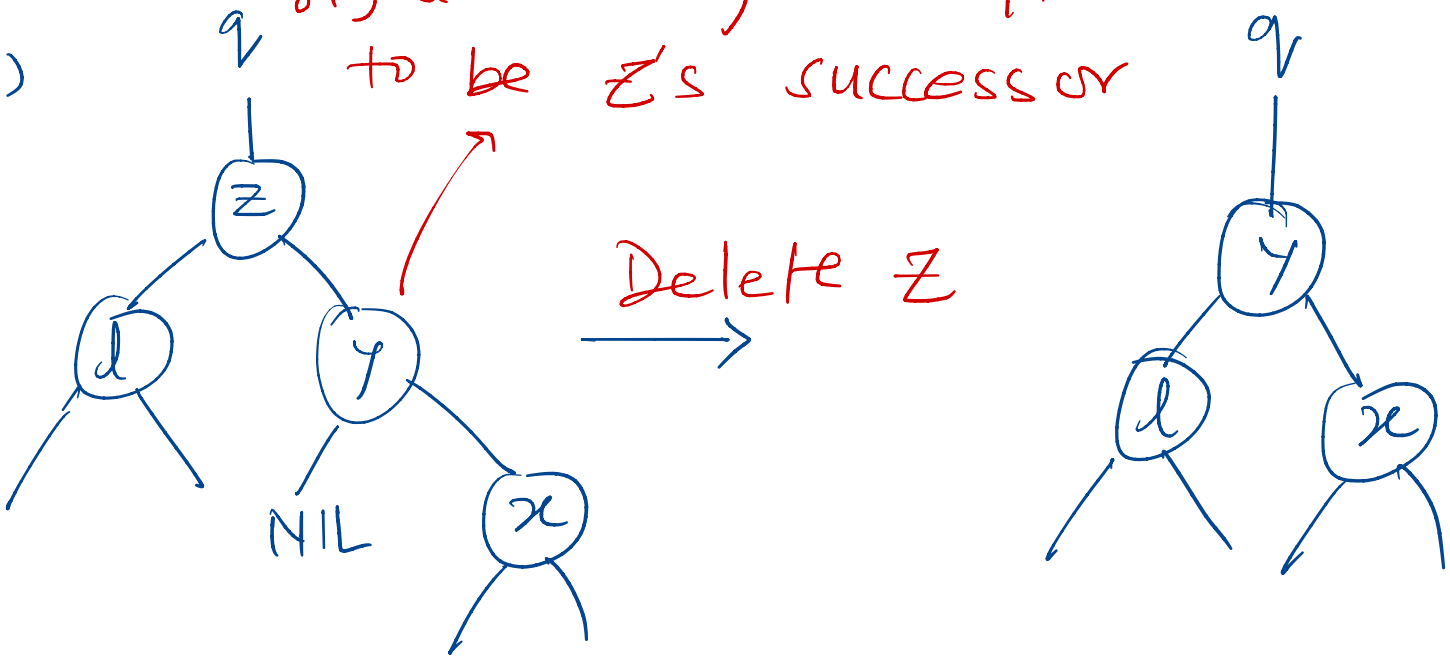
Cases (i) & (ii)

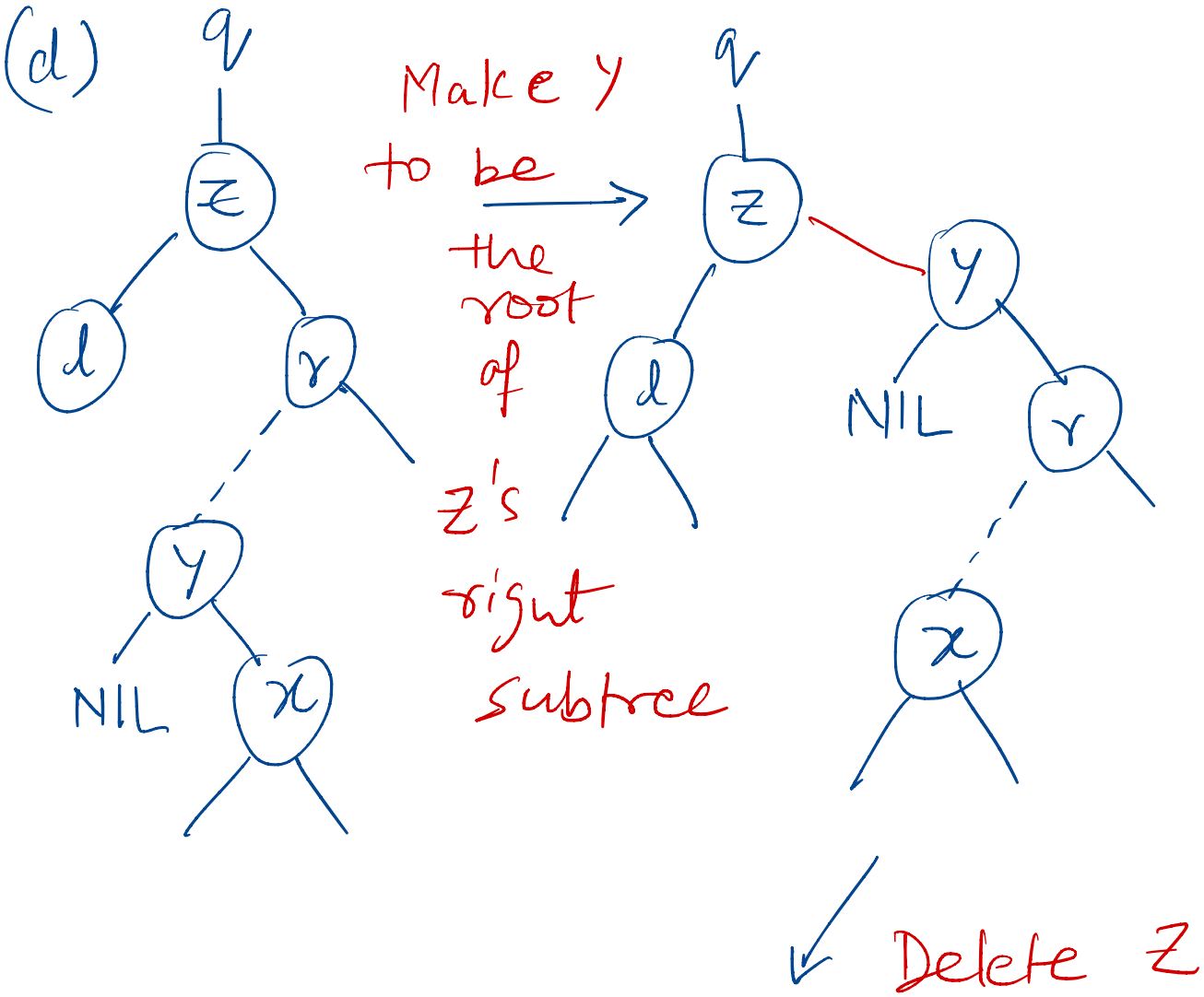
(b)



(c)

right child of z happens to be z's successor




(d)
q
z
l
r
y
NIL
x
Make y to be the root of z's right subtree
q
y
l
r
x
Delete z

- As part of deleting a node, subtrees need to move around in the BST

↳ Define a routine Transplant that replaces one subtree as a child of its parent with another subtree

Transplant: Replaces subtree rooted at node u with subtree rooted at node v.

Node u's parent becomes node v's parent and u's parent ends up having v as an appropriate child.

Transplant (T, u, v)

```
if u.p == NIL
    T.root = v
else if u == u.p.left
    u.p.left = v
else
    u.p.right = v
if v ≠ NIL
    v.p = u.p
```

Tree-Delete (T, z)

```
if z.left == NIL                          replace z by its right child
    Transplant (T, z, z.right)
else if z.right == NIL                    replace z by its left child
    Transplant (T, z, z.left)
else                                      successor of z
    y = Tree_Minimum (z.right)
    if y ≠ z.right                        Is y further down the right subtree?
        Transplant (T, y, y.right)        replace y by its right child
        y.right = z.right                 z's right child becomes y's right child
        y.right.p = y
    Transplant (T, z, y)                  replace z by its successor y
    y.left = z.left                       set y's left to z's left
    y.left.p = y
```